

ESPECIALIZACION EN INTELIGENCIA ARTIFICIAL

Computación en la Nube

INFORME PRACTICA 1

Configuración del Entorno Virtualizado con Vagrant

Estudiante:

Milton Jaramillo

Docente:

Prof. Oscar Mondragon

2026

1. Introducción

La virtualización es una tecnología fundamental en el campo de la computación en la nube, ya que permite crear entornos de ejecución aislados dentro de una misma máquina física. En esta práctica se exploró el uso de Vagrant junto con VirtualBox como herramientas para automatizar la creación y gestión de máquinas virtuales de forma reproducible y sencilla.

Vagrant es una herramienta de código abierto desarrollada por HashiCorp que facilita la configuración de entornos de desarrollo virtualizados mediante un archivo llamado Vagrantfile. Este archivo describe en lenguaje Ruby las características de las máquinas virtuales que se desean crear, como el sistema operativo base (box), la configuración de red y el nombre del host.

El objetivo principal de esta práctica fue configurar un ambiente de desarrollo con dos máquinas virtuales Ubuntu (una actuando como servidor y otra como cliente), verificar la conectividad de red entre ellas, instalar herramientas básicas y, finalmente, publicar el box modificado en Vagrant Cloud para poder reutilizarlo en prácticas futuras.

2. Objetivos

2.1 Objetivo general

Configurar un entorno virtualizado funcional usando Vagrant y VirtualBox, con dos máquinas Ubuntu comunicadas en red privada, como base para las prácticas del curso de Computación en la Nube.

2.2 Objetivos específicos

- Instalar y verificar las versiones de VirtualBox y Vagrant en Windows.
- Crear y configurar un Vagrantfile con dos máquinas virtuales (servidorUbuntu y clienteUbuntu).
- Establecer conectividad de red entre las dos máquinas virtuales.
- Instalar herramientas de red (net-tools) y el editor Vim en ambas máquinas.
- Publicar el box modificado en Vagrant Cloud para su reutilización futura.

3. Herramientas y Tecnologías Utilizadas

Para el desarrollo de esta práctica se utilizaron las siguientes herramientas:

- VirtualBox: Hipervisor de tipo 2 desarrollado por Oracle que permite crear y gestionar máquinas virtuales sobre el sistema operativo anfitrión. En esta práctica funcionó como el proveedor (provider) de Vagrant.
- Vagrant: Herramienta de HashiCorp que automatiza la creación y configuración de máquinas virtuales a través del archivo Vagrantfile. Permite reproducir entornos de forma consistente en cualquier equipo.
- Ubuntu 20.04 (bento/ubuntu-20.04): Sistema operativo Linux en su versión 20.04 LTS, utilizado como imagen base (box) para ambas máquinas virtuales.
- Plugin vagrant-vbguest: Plugin de Vagrant que mantiene actualizadas las VirtualBox Guest Additions dentro de las máquinas virtuales, mejorando la compatibilidad y el rendimiento.
- net-tools: Paquete de utilidades de red para Linux que incluye comandos como ifconfig, netstat y route, indispensables para verificar la configuración de red.
- Vim: Editor de texto en línea de comandos ampliamente usado en sistemas Linux, instalado en ambas máquinas para facilitar la edición de archivos de configuración.
- Vagrant Cloud: Repositorio en la nube de HashiCorp donde se pueden publicar y compartir imágenes (boxes) de máquinas virtuales para reutilizarlas desde cualquier equipo.

4. Descripción del Proceso de Instalación y Configuración

4.1 Instalación de VirtualBox y Vagrant

El primer paso fue descargar e instalar VirtualBox desde su sitio oficial (www.virtualbox.org). Una vez instalado el hipervisor, se descargó Vagrant desde el repositorio de HashiCorp. Se utilizó la versión 2.4.9 para Windows de 64 bits (`vagrant_2.4.9_windows_amd64.msi`).

Tras la instalación, se abrió una consola del sistema (`cmd`) y se ejecutó el comando `vagrant version` para confirmar que Vagrant quedó correctamente instalado. La consola mostró la versión instalada, lo que indicó que el proceso fue exitoso.

Adicionalmente, se instaló el plugin `vagrant-vbguest` con el siguiente comando:

```
vagrant plugin install vagrant-vbguest
```

Fue necesario desactivar Hyper-V en Windows 11 antes de ejecutar Vagrant con VirtualBox, ya que ambos hipervisores generan conflictos al intentar correr simultáneamente. Para ello se ejecutó el siguiente comando en una consola con permisos de administrador y se reinició el equipo:

```
bcdedit /set hypervisorlaunchtype off
```

4.2 Creación del Vagrantfile

Se creó un directorio llamado "Modulo 1" en el equipo local. Dentro de ese directorio se generó el archivo `Vagrantfile` usando el comando `vagrant init` para obtener una plantilla base, y luego se editó para que contuviera únicamente la configuración necesaria para este laboratorio.

El `Vagrantfile` final define dos máquinas virtuales: `servidorUbuntu` con la IP `192.168.100.3` y `clienteUbuntu` con la IP `192.168.100.2`. Ambas utilizan la imagen base `bento/ubuntu-20.04` disponible en Vagrant Cloud. También se incluyó una sección para deshabilitar las actualizaciones automáticas del plugin `vbguest` y evitar problemas durante el arranque. Adicionalmente, se agregó el parámetro `config.vm.boot_timeout = 600` en el `Vagrantfile` para evitar que las máquinas virtuales fallaran por timeout durante su arranque inicial.

4.3 Levantamiento de las máquinas virtuales

Desde la consola de Windows, estando dentro del directorio "Modulo 1", se ejecutó el comando `vagrant up`. Vagrant descargó automáticamente la imagen base de Ubuntu 20.04 desde Vagrant Cloud (la primera vez este proceso puede tardar varios minutos según la velocidad de conexión a internet) y creó las dos máquinas virtuales en VirtualBox.

Una vez terminado el proceso, se verificó el estado de ambas máquinas con el comando `vagrant status`, que confirmó que `servidorUbuntu` y `clienteUbuntu` estaban en estado `running` (en ejecución).

4.4 Configuración interna de las máquinas

Para ingresar a cada máquina virtual se utilizó SSH a través de Vagrant. Para el servidor se ejecutó `vagrant ssh servidorUbuntu`. Dentro de la máquina se escalaron privilegios con `sudo -i` para operar como superusuario y poder instalar paquetes.

Se instalaron las herramientas de red con `apt-get install net-tools` y el editor Vim con `apt-get install vim`. Este mismo procedimiento se repitió en la máquina `clienteUbuntu`.

Para verificar la configuración de red se usó el comando `ifconfig`, que confirmó que cada máquina tenía asignada su IP correctamente dentro de la red privada definida en el `Vagrantfile`. Luego se hizo una prueba de conectividad entre las dos máquinas usando `ping`, comprobando que la comunicación entre servidor y cliente funcionaba correctamente.

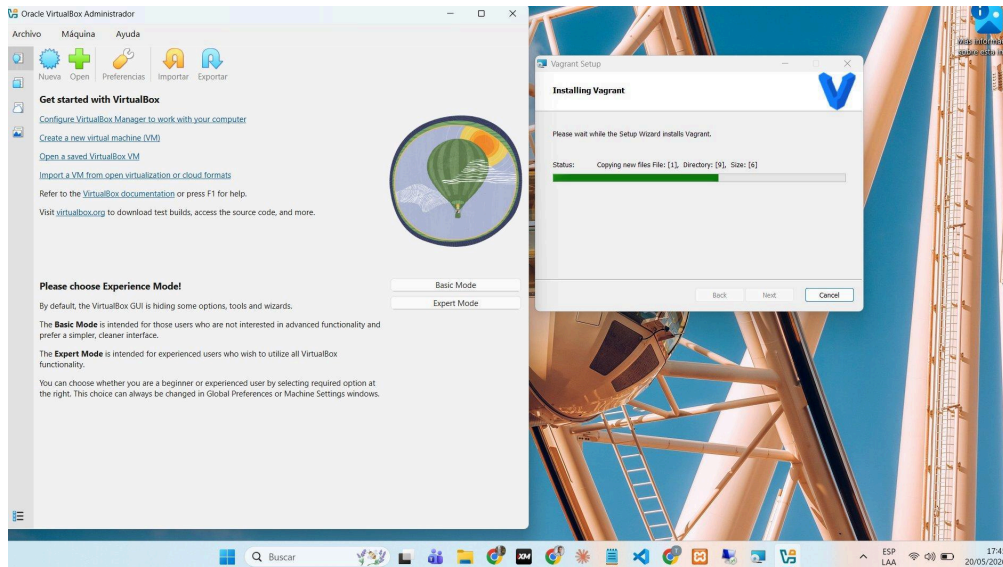
4.5 Ejercicios Linux

Como parte complementaria de la práctica se realizaron los ejercicios Linux de la Guía 2, ejecutados desde la VM servidorUbuntu. Se practicaron los siguientes comandos:

- `pwd`: muestra el directorio de trabajo actual.
- `cd`: permite cambiar de directorio dentro del sistema de archivos.
- `mkdir`: crea nuevos directorios en la ubicación indicada.
- `ls`: lista los contenidos de un directorio.
- `rmdir`: elimina directorios vacíos.
- `pushd` / `popd`: guardan y retornan a ubicaciones previamente almacenadas en la pila de directorios.
- `touch`: crea archivos vacíos o actualiza la fecha de modificación de archivos existentes.
- `cp`: copia archivos o directorios de una ubicación a otra.
- `mv`: mueve o renombra archivos y directorios.
- `less` / `more` / `cat`: permiten leer el contenido de archivos de texto en la terminal.
- `rm`: elimina archivos o directorios del sistema.

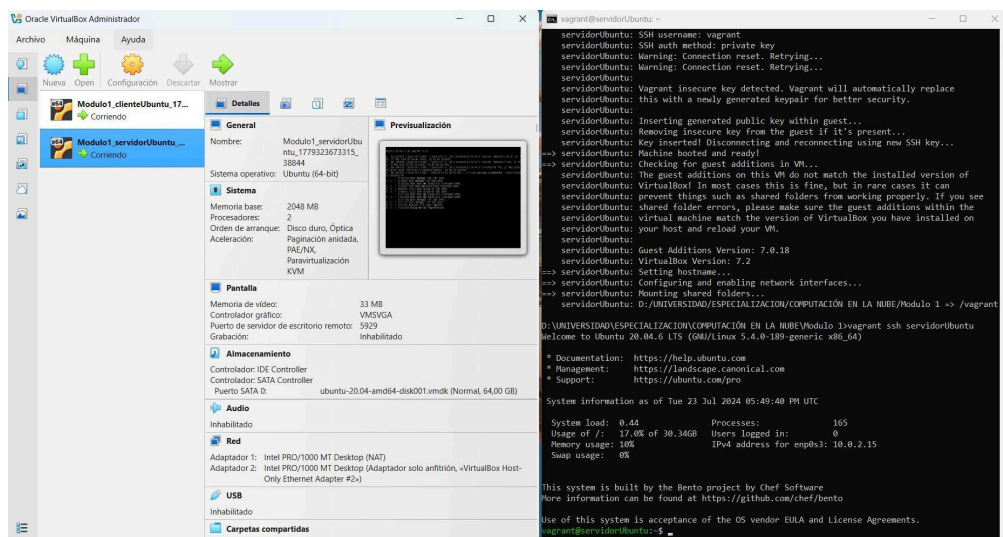
Capturas de Pantalla

Figura 1 - Instalación de VirtualBox y Vagrant en Windows



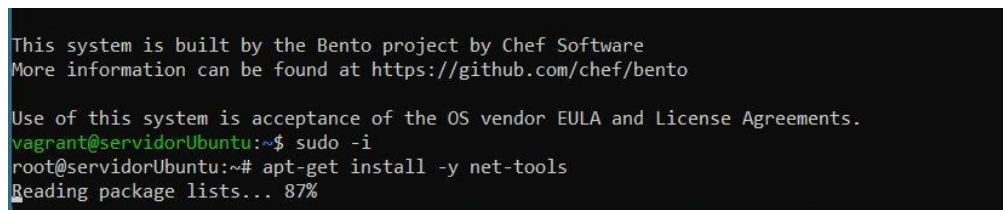
Instalación simultánea de VirtualBox y el asistente de Vagrant en Windows 11

Figura 2 - Levantamiento de las dos VMs con vagrant up



Ejecución de vagrant up: ambas VMs servidorUbuntu y clienteUbuntu corriendo en VirtualBox

Figura 3 - Instalación de net-tools en servidorUbuntu



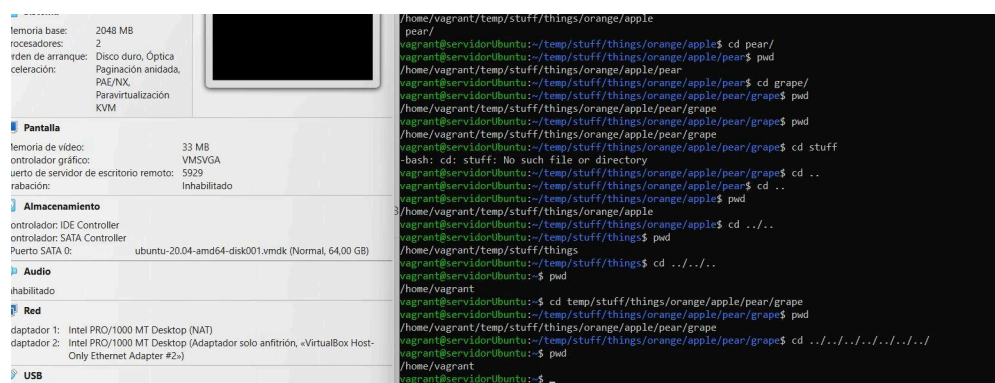
Instalación del paquete net-tools mediante apt-get dentro de la VM servidorUbuntu

Figura 4 - Listado de contenidos con ls (Ejercicios Linux - Guía 2)

```
vagrant@servidorUbuntu:~$ cd temp/
vagrant@servidorUbuntu:~/temp$ ls
stuff
vagrant@servidorUbuntu:~/temp$ cd stuff/
vagrant@servidorUbuntu:~/temp/stuff$ ls
things
vagrant@servidorUbuntu:~/temp/stuff$ cd things/
vagrant@servidorUbuntu:~/temp/stuff/things$ ls
orange
vagrant@servidorUbuntu:~/temp/stuff/things$ cd orange/
vagrant@servidorUbuntu:~/temp/stuff/things/orange$ ls
apple
vagrant@servidorUbuntu:~/temp/stuff/things/orange$ cd apple/
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple$ ls
pear
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple$ cd pear/
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple/pear$ ls
grape
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple/pear$ cd grape/
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple/pear/grape$ ls
vagrant@servidorUbuntu:~/temp/stuff/things/orange/apple/pear/grape$
```

Navegación por directorios anidados usando cd y ls en la VM servidorUbuntu

Figura 5 - Navegación con pwd y cd (Ejercicios Linux - Guía 2)



Uso de pwd, cd ../ y rutas absolutas para navegar la estructura de directorios

Figura 6 - Verificación de IPs con ifconfig

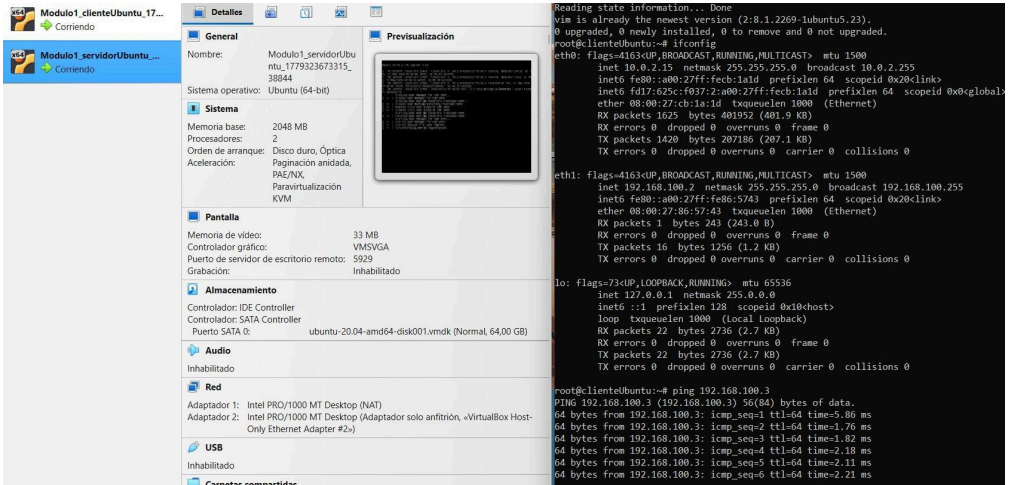

```
oot@servidorUbuntu:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::a00:27ff:feeb:1a1d prefixlen 64 scopeid 0x20<link>
    inet6 fd17:625c:f037:2:a00:27ff:feeb:1a1d prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:cb:1a:1d txqueuelen 1000 (Ethernet)
    RX packets 1578 bytes 377263 (377.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1347 bytes 194307 (194.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.100.3 netmask 255.255.255.0 broadcast 192.168.100.255
    inet6 fe80::a00:27ff:fe52:9a2d prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:52:9a:2d txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14 bytes 1116 (1.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 18 bytes 2268 (2.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18 bytes 2268 (2.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

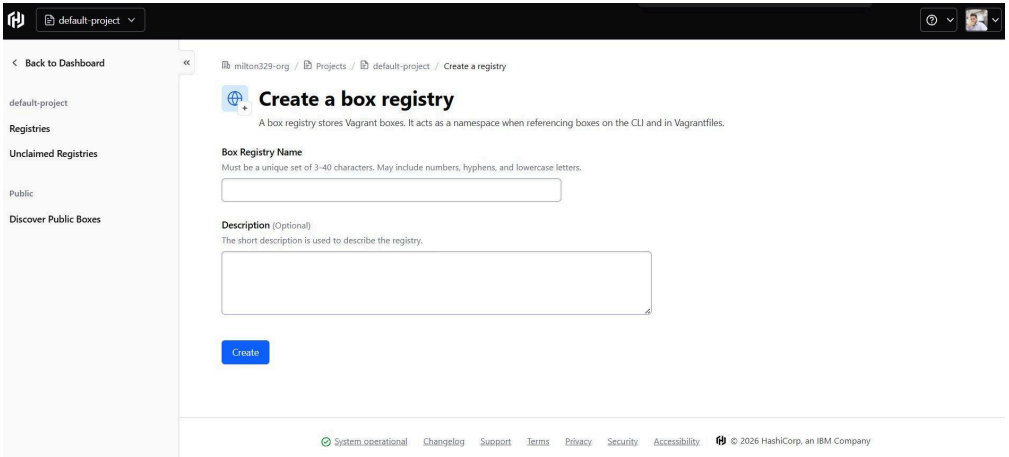
Salida de ifconfig en servidorUbuntu mostrando la IP 192.168.100.3 en la interfaz eth1

Figura 7 - Prueba de conectividad con ping entre VMs



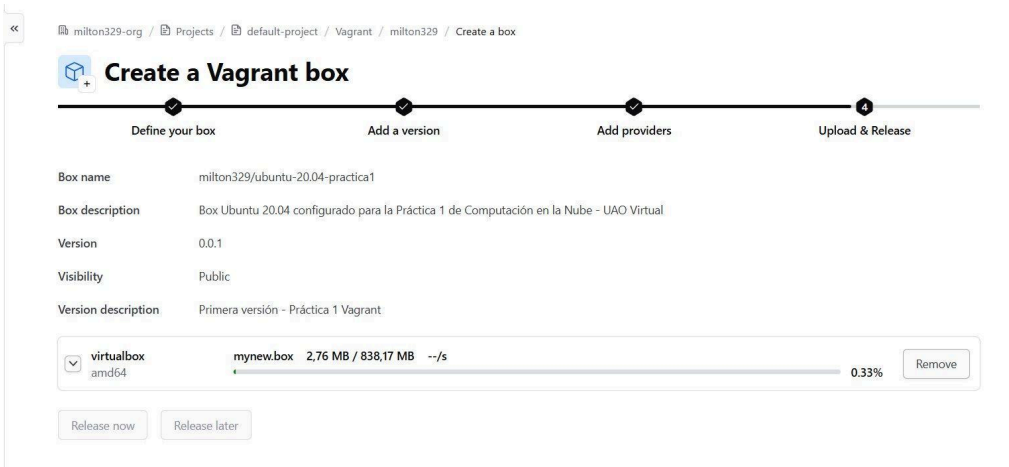
Ping exitoso desde clienteUbuntu (192.168.100.2) hacia servidorUbuntu (192.168.100.3)

Figura 8 - Creacion del Box Registry en HCP Vagrant



Formulario de creacion del registry en app.vagrantup.com para el proyecto default-project

Figura 9 - Subida del box a HCP Vagrant Registry



Carga del archivo mynew.box (838 MB) con proveedor virtualbox/amd64 en proceso de subida

Figura 10 - Version 0.0.1 publicada y liberada en Vagrant Cloud

The screenshot displays the Vagrant Cloud interface for the box `milton329/ubuntu-20.04-practica1`. The main section shows version **0.0.1** with a green **Released** status. A note indicates it was created 12 minutes ago. Below this, the **Providers** section lists `virtualbox` with an `amd64` architecture and a `default` provider, hosted on HCP Vagrant Registry (838,17 MB). On the right, a sidebar titled **How to use this box with Vagrant** provides instructions. **Step 1** shows two options: Option 1 is to create a Vagrantfile and initiate the box using the command `vagrant init milton329/ubuntu-20.04-practica1 --box-version 0.0.1`. Option 2 is to open the Vagrantfile and replace the contents with the following code block:

```
Vagrant.configure("2") do |config|
  config.vm.box = "milton329/ubuntu-20.04-practica1"
  config.vm.box_version = "0.0.1"
end
```

Version 0.0.1 del box `milton329/ubuntu-20.04-practica1` con estado Released en HCP Vagrant Registry

5. Publicación del Box en Vagrant Cloud

Una vez configuradas y verificadas las máquinas virtuales, se procedió a empaquetar la máquina servidorUbuntu como un nuevo box para poder reutilizarla en el futuro. Desde la consola se ejecutó el siguiente comando:

```
vagrant package servidorUbuntu --output mynew.box
```

Este comando generó un archivo llamado mynew.box con el estado actual de la máquina virtual. Luego se agregó al repositorio local de Vagrant con el comando `vagrant box add mynewbox mynew.box`, lo que permite referenciarlo desde cualquier Vagrantfile en el equipo.

Para publicarlo en Vagrant Cloud se accedió al portal app.vagrantup.com, se creó un nuevo box con nombre y descripción, se definió la versión 0.0.1, se seleccionó VirtualBox como proveedor y se cargó el archivo mynew.box. Finalmente, se liberó la versión usando la opción "release" para que quedara disponible públicamente.

El box quedó publicado en HCP Vagrant Registry y está disponible públicamente en: <https://app.vagrantup.com/milton329/boxes/ubuntu-20.04-practica1>

6. Conclusiones

Esta práctica permitió entender de manera práctica como funciona la virtualización en el contexto de la computación en la nube. Herramientas como Vagrant simplifican enormemente el proceso de crear y gestionar entornos virtuales, ya que con un solo archivo de configuración es posible reproducir el mismo entorno en cualquier máquina, algo muy útil tanto en proyectos de desarrollo como en ambientes educativos.

Uno de los aspectos más interesantes fue la configuración de la red privada entre las dos máquinas virtuales, ya que ilustra de forma directa el modelo cliente-servidor que es la base de gran parte de los servicios en la nube. La verificación de conectividad mediante ping y la consulta de IPs con ifconfig confirmaron que el entorno quedó correctamente configurado.

La experiencia de publicar un box en Vagrant Cloud también fue muy enriquecedora, ya que introduce el concepto de repositorios de imágenes de máquinas virtuales, algo equivalente a registros de contenedores como Docker Hub. Esto sienta una base conceptual importante para entender como se distribuyen y reutilizan imágenes en entornos de nube reales.

En conclusión, esta práctica fue una introducción sólida al mundo de la virtualización con Vagrant y abre el camino hacia temas más avanzados del curso como la contenerización y los servicios en la nube.

7. Referencias

- HashiCorp. (2026). Vagrant Documentation. <https://www.vagrantup.com/>
- HashiCorp. (2026). Vagrant Cloud - Creating a New Box. <https://atlas.hashicorp.com/help/vagrant/boxes/create>
- Oracle. (2026). VirtualBox Official Site. <https://www.virtualbox.org/>
- GitHub. (2026). GitHub - Repositorio de control de versiones. <https://github.com/>
- Git SCM. (2026). Git Tutorial. <https://git-scm.com/docs/gittutorial>
- Mondragon, O. (2026). Guia 1 - Configuración entorno virtualizado con Vagrant. UAO Virtual.
- Jaramillo, M. (2026). Repositorio compunube. GitHub. <https://github.com/milton329/compunube>